

```

"""
LAB 1: Display Density Metrics & Image
Array Mechanics in NumPy
Solves Tasks 1-4. Run each task's function,
or run the file directly
to execute all of them in order.
"""

import math
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

# -----
# -----
# TASK 1: Display Pixel Density (PPI/DPI)
# Calculator
# -----
# -----

def task1_dpi_calculator(w_px=None,
h_px=None, diag_in=None):
    """
    Computes total pixels, aspect ratio,
    DPI, and density category.
    If no arguments are passed, prompts the
    user for input.
    """
    if w_px is None:
        w_px = int(input("Enter horizontal

```

```

resolution (pixels): ")
    if h_px is None:
        h_px = int(input("Enter vertical
resolution (pixels): "))
    if diag_in is None:
        diag_in = float(input("Enter
physical diagonal size (inches): "))

    # Total pixel count
    total_pixels = w_px * h_px

    # Aspect ratio, simplified using GCD
    divisor = math.gcd(w_px, h_px)
    ratio_w, ratio_h = w_px // divisor,
h_px // divisor

    # DPI / PPI = diagonal pixel count /
diagonal inches
    diagonal_px = math.sqrt(w_px ** 2 +
h_px ** 2)
    dpi = diagonal_px / diag_in

    # Density classification
    if dpi < 100:
        category = "Low Density (Standard
Monitor)"
    elif dpi <= 200:
        category = "Medium Density (HD
Display)"
    else:
        category = "High Density (Retina /
Mobile)"

    print("\n--- DISPLAY METRICS ANALYSIS -

```

```

--")
    print(f"Total Pixel Count :
{total_pixels:,} pixels")
    print(f"Aspect Ratio      : {ratio_w}:
{ratio_h}")
    print(f"Calculated DPI    : {dpi:.2f}
DPI")
    print(f"Density Category   :
{category}")

    return {
        "total_pixels": total_pixels,
        "aspect_ratio": f"{ratio_w}:
{ratio_h}",
        "dpi": round(dpi, 2),
        "category": category,
    }

# -----
# -----
# TASK 2: Environment Setup & Synthetic
Image Matrix Creation
# -----
# -----

def task2_synthetic_matrix():
    """
    Builds a 300x400x3 uint8 array and
    fills each quadrant with a color.
    """
    H, W, C = 300, 400, 3
    img = np.zeros((H, W, C),
dtype=np.uint8)

```

```

        mid_h, mid_w = H // 2, W // 2

        img[0:mid_h, 0:mid_w] = [255, 0,
0]         # Top-left: Red
        img[0:mid_h, mid_w:W] = [0, 255,
0]         # Top-right: Green
        img[mid_h:H, 0:mid_w] = [0, 0,
255]        # Bottom-left: Blue
        img[mid_h:H, mid_w:W] = [255, 255,
255]        # Bottom-right: White

        print("\n--- SYNTHETIC MATRIX METRICS -
        --")
        print(f"Array Shape (H, W, C) :
        {img.shape}")
        print(f"Data Type                :
        {img.dtype}")
        print(f"Total Elements            :
        {img.size:,} values")
        print(f"Memory Footprint          :
        {img.nbytes:,} bytes ({img.nbytes/1024:.2f}
        KB)")

        # Optional: view the result
        plt.imshow(img)
        plt.title("Task 2: Four-Quadrant
        Synthetic Image")
        plt.axis("off")

        plt.savefig("/home/claude/task2_quadrants.p
        ng", dpi=100, bbox_inches="tight")
        plt.close()

        return img

```

```

# -----
# TASK 3: Channel Slicing & Isolation
# -----

def
task3_channel_extraction(image_path="sample
.jpg"):
    """
        Loads an image, isolates R/G/B
        channels, and plots a 2x3 grid:
        top row = single-channel color images,
        bottom row = grayscale intensity.
    """
    img =
np.array(Image.open(image_path).convert("RG
B"))

    red_channel = img[:, :, 0]
    green_channel = img[:, :, 1]
    blue_channel = img[:, :, 2]

    # Build single-channel-active color
    images
    red_only = np.zeros_like(img)
    red_only[:, :, 0] = red_channel

    green_only = np.zeros_like(img)
    green_only[:, :, 1] = green_channel

    blue_only = np.zeros_like(img)
    blue_only[:, :, 2] = blue_channel

```

```
        print("\n--- CHANNEL EXTRACTION SUMMARY
        ---")
        print(f"Original Image Shape :
        {img.shape}")
        print(f"Red Channel 2D Shape :
        {red_channel.shape} | Mean Intensity:
        {red_channel.mean():.2f}")
        print(f"Green Channel 2D Shape:
        {green_channel.shape} | Mean Intensity:
        {green_channel.mean():.2f}")
        print(f"Blue Channel 2D Shape :
        {blue_channel.shape} | Mean Intensity:
        {blue_channel.mean():.2f}")

        fig, axes = plt.subplots(2, 3, figsize=
        (12, 8))
        axes[0, 0].imshow(red_only); axes[0,
        0].set_title("Red Only"); axes[0,
        0].axis("off")
        axes[0, 1].imshow(green_only); axes[0,
        1].set_title("Green Only"); axes[0,
        1].axis("off")
        axes[0, 2].imshow(blue_only); axes[0,
        2].set_title("Blue Only"); axes[0,
        2].axis("off")

        axes[1, 0].imshow(red_channel,
        cmap="gray"); axes[1, 0].set_title("Red
        Intensity"); axes[1, 0].axis("off")
        axes[1, 1].imshow(green_channel,
        cmap="gray"); axes[1, 1].set_title("Green
        Intensity"); axes[1, 1].axis("off")
        axes[1, 2].imshow(blue_channel,
```

```

cmap="gray"); axes[1, 2].set_title("Blue
Intensity"); axes[1, 2].axis("off")

plt.tight_layout()

plt.savefig("/home/claude/task3_channels.png", dpi=100, bbox_inches="tight")
plt.close()

print("Display Window : Matplotlib 2x3
Subplot Grid Rendered.")

# -----
# -----
# TASK 4: Spatial Downsampling & Pixelation
via Striding
# -----
# -----

def
task4_downsampling(image_path="sample.jpg",
n=8):
    """
        Downsamples an image using array
striding, then re-expands it with
np.repeat to show the pixelation
effect. Prints memory/size stats.
    """
    img =
np.array(Image.open(image_path).convert("RGB"))
    original_shape = img.shape
    original_bytes = img.nbytes

```

```

    downsampled = img[:, :n, :n, :]
    down_shape = downsampled.shape
    down_bytes = downsampled.nbytes

    # Re-expand: repeat each pixel n times
    # along axis 0 and axis 1
    re_expanded =
    np.repeat(np.repeat(downsampled, n,
    axis=0), n, axis=1)

    # Per-axis reduction (matches the
    # handout's "per axis" wording):
    # each axis shrinks by a factor of n,
    # so reduction = (1 - 1/n) * 100
    dim_reduction_pct = (1 - 1 / n) * 100
    mem_savings_pct = (1 - down_bytes /
    original_bytes) * 100

    print(f"\n--- DOWNSAMPLING ANALYSIS (N
    = {n}) ---")
    print(f"Original Shape      :
    {original_shape} | Memory:
    {original_bytes:,} bytes")
    print(f"Downsampled Shape   :
    {down_shape} | Memory: {down_bytes:,}
    bytes")
    print(f"Re-expanded Shape   :
    {re_expanded.shape} | Visual: Blocky
    Pixelation")
    print(f"Dimension Reduction:
    {dim_reduction_pct:.2f}% reduction per
    axis")
    print(f"Memory Savings      :
    {mem_savings_pct:.2f}% data reduction")

```

```

        fig, axes = plt.subplots(1, 2, figsize=
(10, 5))
        axes[0].imshow(img);
axes[0].set_title("Original");
axes[0].axis("off")
        axes[1].imshow(re_expanded);
axes[1].set_title(f"Pixelated (N={n})");
axes[1].axis("off")
        plt.tight_layout()

plt.savefig("/home/claude/task4_pixelation.
png", dpi=100, bbox_inches="tight")
plt.close()

# -----
# -----
# Helper: makes a fake sample.jpg so Tasks
3 & 4 run even without your own photo
# -----
# -----
def _make_demo_image(path="sample.jpg",
h=1080, w=1920):
    rng = np.random.default_rng(0)
    gradient = np.zeros((h, w, 3),
dtype=np.uint8)
    x = np.linspace(0, 255, w)
    y = np.linspace(0, 255, h)
    gradient[:, :, 0] = x[np.newaxis, :]
    gradient[:, :, 1] = y[:, np.newaxis]
    gradient[:, :, 2] = 128
    Image.fromarray(gradient).save(path)

```

```
if __name__ == "__main__":
    # --- Task 1 demo (using the sample
    values from the handout) ---
    task1_dpi_calculator(1920, 1080,
24)    # Desktop monitor
    task1_dpi_calculator(1170, 2532,
6.1)    # Smartphone screen

    # --- Task 2 ---
    task2_synthetic_matrix()

    # --- Tasks 3 & 4 need a real image;
    using a generated demo image here ---

    _make_demo_image("/home/claude/sample.jpg")

    task3_channel_extraction("/home/claude/sample.jpg")

    task4_downsampling("/home/claude/sample.jpg", n=8)

    print("\nAll tasks complete. Check the
    saved PNGs for the plots.")
```